PLAYWRIGHT CAPSTONE PROJECT REPORT

By: Madhur Kakkar

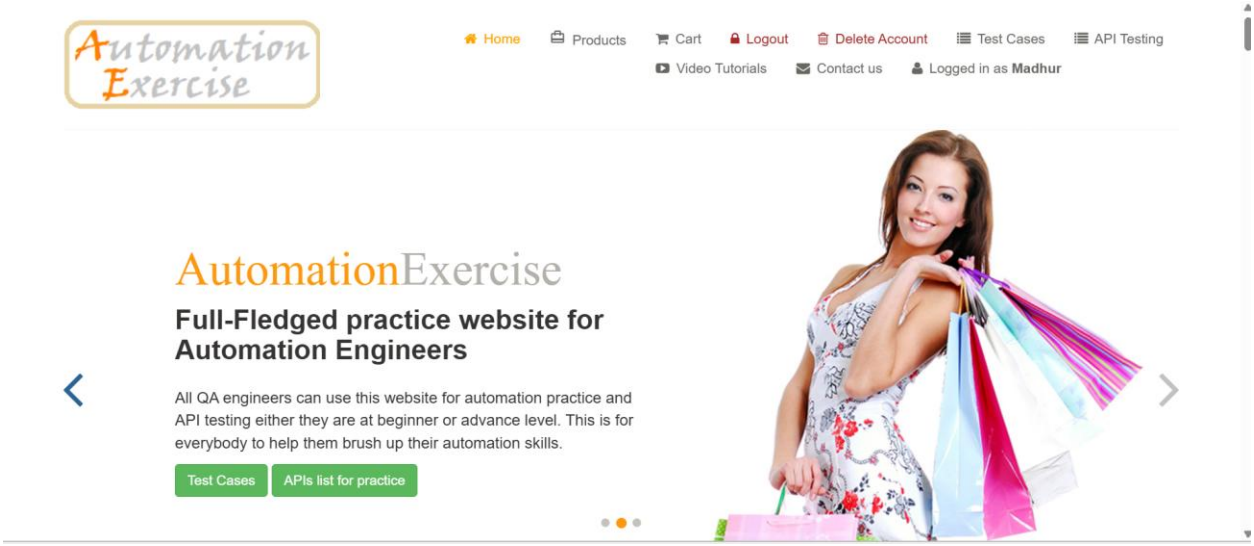
Batch:2
Trainer: Aaryan Kumar
Project: Automation Exercise Testing
GitHub Repository: https://github.com/Madhur373/Capstone_Project_Madhur

Table of Contents

Overview

This project presents an enterprise-level automation testing framework developed with Playwright and JavaScript to validate critical workflows of the Automation Exercise e-commerce application. The framework incorporates both UI and API automation, ensuring comprehensive test coverage across the application.

By leveraging modern automation practices, the solution enables cross-browser execution, detailed test reporting with Allure, and automated test orchestration through GitHub Actions. The framework is designed with a strong focus on scalability, maintainability, reusability, and reliability, aligning with industry standards for modern software quality assurance.



Project Objectives

- Build a scalable Playwright automation framework.
- Automate critical e-commerce workflows.
- Implement UI and API test automation.
- Generate detailed execution reports.
- Integrate automated testing into a CI/CD pipeline.
- Ensure cross-browser compatibility.

Technology Used

- Playwright
- JavaScript
- Typescript
- Node.js
- Git & GitHub
- GitHub Actions
- Allure Reports
- Playwright HTML Reports
- Visual Studio Code

Key Components

- **Page Objects:** Encapsulate locators and reusable page actions.
- **Test Scripts:** Contain business test scenarios and validations.
- **Utilities:** Common helper functions and reusable methods.
- **Configuration Files:** Manage browser settings and execution parameters.
- **Reporting:** Allure Reports and Playwright HTML Reports.
- **CI/CD Pipeline:** GitHub Actions workflow for automated execution.

Requirements

The project required Node.js, Playwright, JavaScript, and GitHub for version control and CI/CD execution. The testing framework was expected to run automated tests in Chromium browser and generate execution reports for analysis.

The framework required test cases to be organized module-wise so that different application areas could be tested independently. Separate test suites were created for authentication, product catalog, cart, checkout, payment, profile, API testing, and homepage validation.

The project also required proper reporting support. Playwright HTML reports, screenshots, videos, traces, and test result artifacts were included to help analyze failed test cases. GitHub Actions was configured to execute the test suite automatically on code push, pull request, and manual workflow trigger.

GitHub Actions Workflow Execution

Figure: GitHub Actions Workflow Execution

```
Capstone_Project_Madhur/
|
+-- .github/
|   +-- workflows/
|      +-- playwright.yml
+-- data/
+-- Docs/
|   +-- Capstone Project.docx
+-- pages/
|   +-- product.page.js
+-- tests/
|   +-- api-addon/
|   +-- auth/
|   +-- cart/
|   +-- catalog/
|   +-- checkout/
|   +-- payments/
|   +-- profile/
+-- allure-results/
+-- allure-report/
+-- playwright-report/
+-- test-results/
+-- playwright.config.js
+-- package.json
+-- package-lock.json
```

Test Design Methodology

The test design methodology for this project follows a modular and scenario-based testing approach. Test cases were designed according to the main business workflows of the e-commerce application, such as authentication, product catalog, cart, checkout, payment, profile, and API validation.

Each module was divided into smaller test scenarios to validate both positive and negative behavior. For example, the authentication module includes valid login, invalid login, empty field validation, wrong password validation, logout, and signup validation. This helped ensure that the application was tested from both user success and error-handling perspectives.

The framework uses Playwright with JavaScript to automate UI and API test cases. UI tests were designed using stable locators, page elements, and reusable helper methods. Common actions such as login, navigation, adding products to cart, checkout setup, and payment form handling were moved into helper functions to reduce duplication and improve maintainability.

Data-driven testing was used for login validation, where multiple credential combinations were tested using structured test data. API tests were also added to validate backend responses such as product list, brand list, search product, and login verification APIs.

The tests were organized folder-wise based on functionality. This made the framework easier to understand, maintain, and execute module by module. GitHub Actions was used for CI/CD execution, and Playwright reports, screenshots, videos, and traces were used for result analysis and debugging.

Modules Covered & Test Scenarios

Module	What Has Been Tested	Key Test Scenarios	Test Cases
Authentication	This module validates user login, logout, signup, and authentication form behavior. It checks both positive and negative login flows, field validations, logged-in user state, and data-driven login coverage.	Valid login, invalid login, empty credential validation, wrong password validation, signup validation, existing user signup validation, logout validation, logged-in navbar validation, data-driven login testing.	23
Product Catalog	This module verifies product listing, search, category, brand, and product detail page functionality. It also validates product visibility, metadata, quantity field, and add-to-cart behavior from listing and detail pages.	Product page visibility, product search, product details validation, product metadata validation, product quantity validation, product category validation, product brand validation, add-to-cart from product listing, add-to-cart from product details.	29
Cart	This module validates cart operations for logged-in and guest users. It checks adding products, removing products, cart item details, cart navigation, empty cart behavior, and checkout access from the cart page.	Add single product to cart, add multiple products to cart, remove product from cart, empty cart validation, guest cart validation, cart item name, price, quantity, and total validation, product detail navigation from cart, checkout navigation from cart, cart persistence after navigation.	19
Checkout	This module verifies the checkout workflow after products are added to the cart. It validates checkout page visibility, delivery and billing address sections, order summary, comment field, guest checkout restrictions, and order flow navigation.	Checkout page visibility, delivery address validation, billing address validation, order summary validation, comment field validation, place order navigation, guest checkout modal validation, end-to-end checkout flow.	22
Payment	This module validates payment page UI and payment form	Payment page visibility, payment form field validation, card input	23

	behavior. It checks card input fields, required-field validations, successful payment flow, payment confirmation page, and post-payment navigation.	validation, browser required-field validation, successful payment confirmation, payment done URL validation, continue button navigation, cart empty validation after order completion.	
Profile	This module validates logged-in user profile-related UI and account information behavior. It checks username visibility, session persistence, logout link visibility, hidden signup/login link state, and account information form fields.	Logged-in username visibility, logout link visibility, signup/login link hidden after login, session persistence, account information form validation, date of birth dropdown validation.	11
API Testing - Add on	This module validates backend API responses from the Automation Exercise application. It checks product APIs, brand APIs, search API, login verification API, response status behavior, and response body data validation.	Get all products API validation, unsupported products API method validation, get all brands API validation, unsupported brands API method validation, search product API validation, search product without parameter validation, verify login API validation, invalid login API validation, product response body validation, brand response body validation.	11
Home / UI Validation-Add on	This module validates that the application homepage loads successfully and confirms basic UI availability before deeper workflow testing.	Homepage load validation and base page accessibility validation.	1

Test Cases

Authentication

authValidation.spec.js
Contains 4 test cases for invalid login, empty credentials, wrong password, and already registered signup email validation.

loginFormCheck.spec.js
Contains 9 test cases for login page title, login form visibility, email/password fields, login button, signup button, signup heading, subscription section, and contact link validation.

userLogin.spec.js
Contains 4 test cases for valid login, logout, logout link visibility, and already logged-in user redirection.

signup.spec.js
Contains 2 test cases for new user signup navigation and signup name field input validation.

loginDataDriven.spec.js
Contains data-driven login test cases for valid credentials, wrong password, non-existing email, and empty email browser validation.

Product Catalog

productVisibility.spec.js
Contains 6 test cases for products page URL, page title, all products heading, product cards, product name/price, and product image visibility.

productSearch.spec.js
Contains 6 test cases for search input, search button, valid search, searched products heading, no-result search, and clearing/reusing search.

productDetail.spec.js
Contains 6 test cases for product detail navigation, product name, category, price, availability, and quantity field validation.

productCategory.spec.js
Contains 4 test cases for category sidebar, Women category expansion, Dress filter, and Men category expansion.

productBrand.spec.js
Contains 2 test cases for brand sidebar visibility and brand filter URL validation.

productAddToCart.spec.js
Contains 5 test cases for add-to-cart button visibility, cart modal, continue shopping button, view cart link, and product detail add-to-cart flow.

Shopping Cart

cartVisibility.spec.js
Contains 5 test cases for cart URL, cart table visibility, proceed to checkout button, product image, and delete button visibility.

cartItemDetails.spec.js
Contains 5 test cases for cart product name, price, quantity, total price, and product detail link validation.

cartAddRemove.spec.js
Contains 6 test cases for adding one, two, and three products, removing products, empty cart after removal, and duplicate product quantity validation.

cartEmptyAndGuest.spec.js
Contains 3 test cases for empty cart message, guest user add-to-cart, and guest checkout login/register modal.

cartNavigation.spec.js
Contains 3 test cases for checkout navigation, product detail navigation from cart, and cart persistence after navigation.

Checkout

checkoutVisibility.spec.js
Contains 6 test cases for checkout URL, delivery address, billing address, order review table, comment textarea, and place order button visibility.

checkoutAddress.spec.js
Contains 5 test cases for delivery name, delivery country, billing name, billing country, and matching delivery/billing address validation.

checkoutOrderSummary.spec.js
Contains 5 test cases for order summary row, product name, price, quantity, and total price visibility.

checkoutInteraction.spec.js
Contains 4 test cases for checkout comment input, place order navigation, guest checkout modal, and register/login link navigation.

checkoutE2E.spec.js
Contains 2 test cases for continue button redirection after order confirmation and cart empty validation after successful order.

Payment

paymentFormVisibility.spec.js
Contains 8 test cases for payment page navigation, payment heading, name on card, card number, CVC, expiry month, expiry year, and pay button visibility.

paymentFieldInput.spec.js
Contains 6 active test cases for payment field input behavior including name, card number, CVC, expiry year, filling all fields, and clearing/re-entering name.

paymentValidation.spec.js
Contains 5 test cases for required-field validation on empty payment form, missing card number, missing CVC, missing expiry month, and missing expiry year.

paymentCheckoutFlow.spec.js
Contains 4 test cases for checkout delivery address, order summary before payment, checkout comment field, and guest checkout restriction.

paymentSuccess.spec.js
Contains 5 test cases for successful payment message, payment done URL, continue button visibility, redirect to home, and cart empty after order.

Profile

profileVisibility.spec.js
Contains 5 test cases for logged-in username visibility, logout link visibility, signup/login link hidden state, username text validation, and session persistence.

profileAccountInfo.spec.js
Contains 6 test cases for account information heading, title radio buttons, pre-filled name, pre-filled email, password field, and date of birth dropdowns.

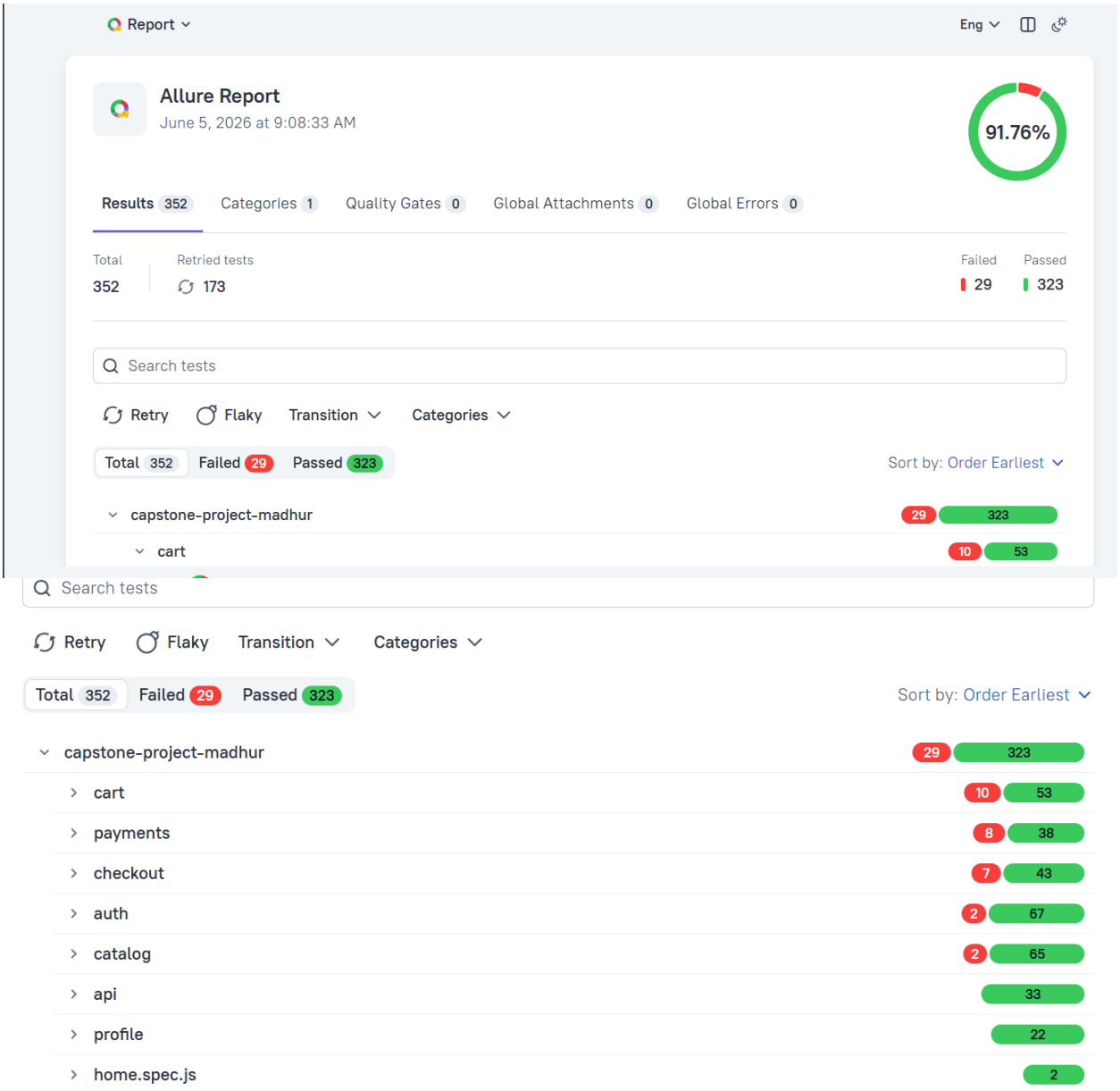
API Testing - Add On

automationExerciseApi.spec.js
Contains 11 API test cases for products list, unsupported products method, brands list, unsupported brands method, product search, search without parameter, login verification without email, delete login verification, invalid login, product response body, and brand response body validation.

Home / UI

home.spec.js
Contains 1 test case to validate that the homepage loads successfully.

Allure Dashboard Overview



This screenshot demonstrates the Allure reporting dashboard displaying:

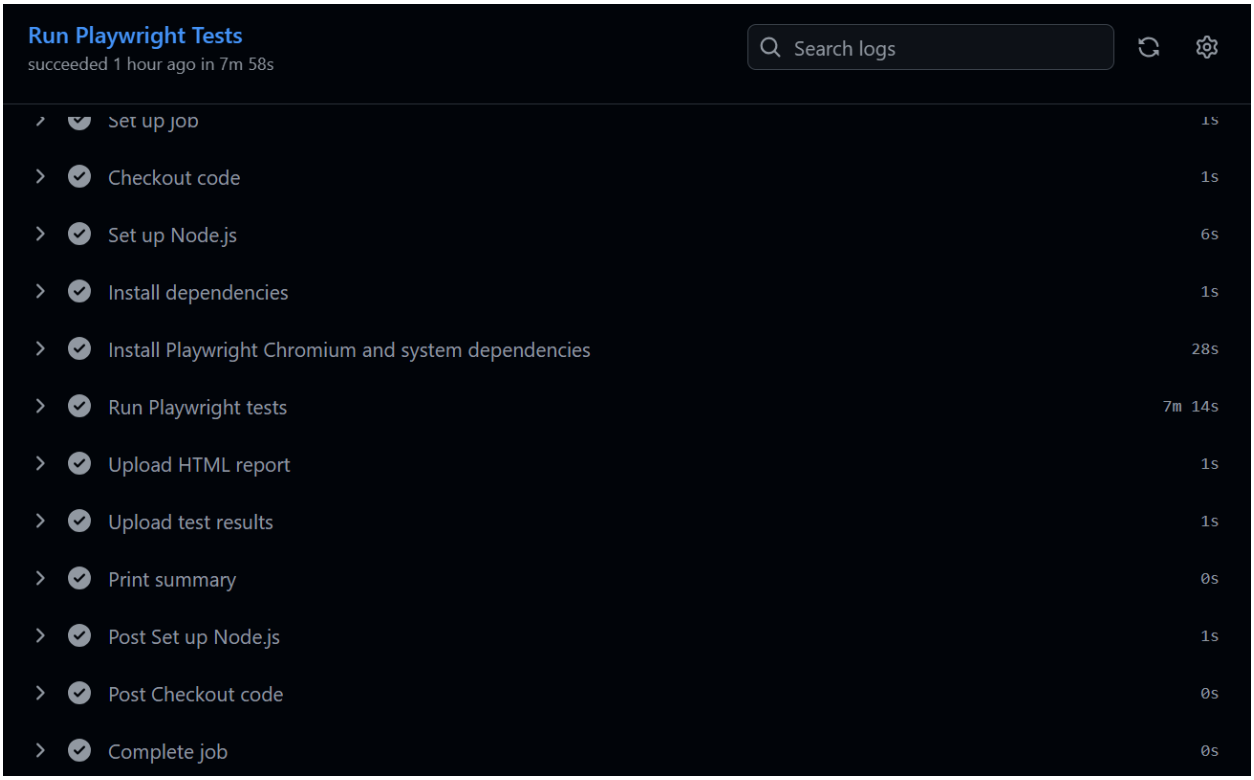
- Test Execution Summary
- Pass/Fail Statistics
- Environment Information
- Test Categories
- Execution Trends

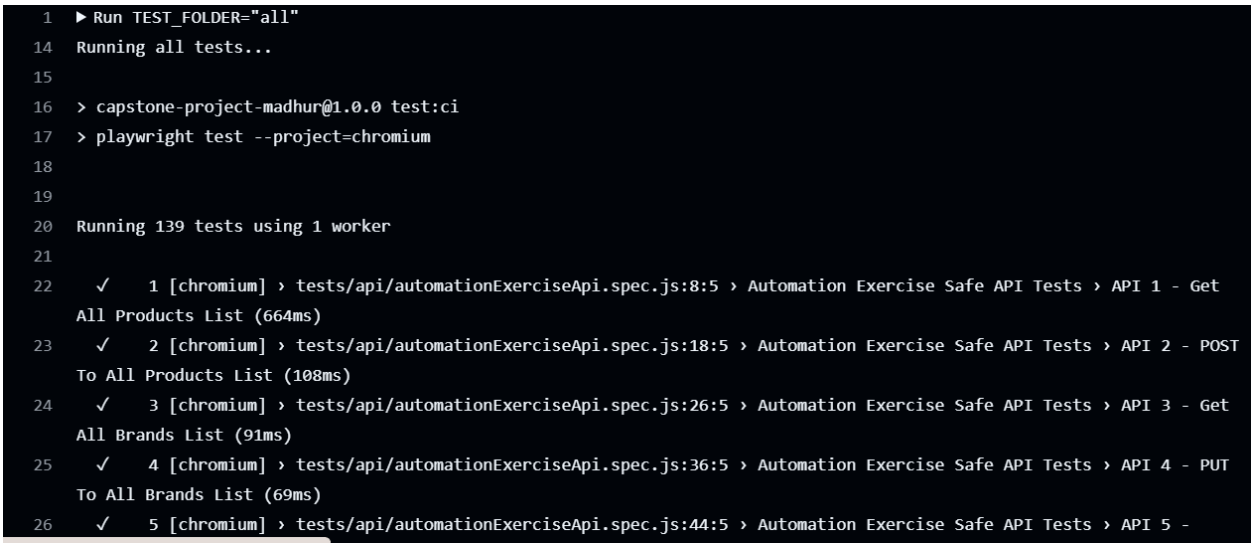
Execution Results

The framework was successfully executed across multiple browsers including:

- Chromium
- Firefox
- WebKit

CI/Cd Execution





Automated UI and API test suites were executed successfully. Reports were generated using Allure and Playwright HTML Reporter. GitHub Actions workflows automated the testing process and ensured consistent execution across environments.

Project Statistics

Metric	Value
Total Test Cases	139
UI Test Cases	128
API Test Cases	11
Browsers Supported	3
Reports Generated	2
CI/CD Pipeline	GitHub Actions
Framework Pattern	POM

Challenges Faced

One major challenge was handling application stability during test execution, especially on GitHub Actions. The target application loads third-party scripts and advertisements, which sometimes caused delays or page loading issues in headless browser execution. To handle this, helper functions were used for safer navigation and blocking unnecessary ad or tracker requests.

Another challenge was maintaining test reliability when multiple test cases used the same test account and cart flow. Since cart, checkout, and payment workflows depend on user session and product state, tests had to be executed carefully with controlled setup steps and a single worker configuration.

GitHub Actions setup also required troubleshooting. Browser installation and Playwright execution needed proper configuration so that Chromium could be installed and detected correctly in the CI environment. The workflow was updated to install dependencies, install Playwright Chromium with system dependencies, and upload test artifacts after execution.

Some test cases also failed due to dynamic website behavior, changed UI text, or differences between local and CI execution. These unstable test cases were reviewed and removed from the final executable suite to keep the automation run stable and consistent.

Conclusion

This capstone project successfully demonstrates modern test automation practices using Playwright, JavaScript, and CI/CD integration. The framework provides reliable, maintainable, and scalable automation solutions for e-commerce applications while following industry best practices.

The project highlights practical implementation of automated UI and API testing, advanced reporting mechanisms, and continuous integration workflows, making it suitable for enterprise-level software quality assurance processes.